

Synchronization & clocks

- A distributed system is a collection of computers that are spatially separated and **do not share** a **common memory**.
- The processes executing on these computers communicate with one another by exchanging messages over communication channels and the messages are delivered after an arbitrary transmission **delay**.
 - we mainly concentrate on how processes can synchronize.
- It is important that multiple processes do not simultaneously access a shared resource, such as printer, but instead cooperate in granting each other **temporary** exclusive access.
 - Another example is that multiple processes may sometimes need to agree on the ordering of events, such as whether message m_1 from process P was sent before or after message m_2 from process Q .

Fundamental Limitations of a Distributed System

- Distributed systems are assumed to have **no global clock**
 - Impact of the absence of global time: order of event can not be determined.

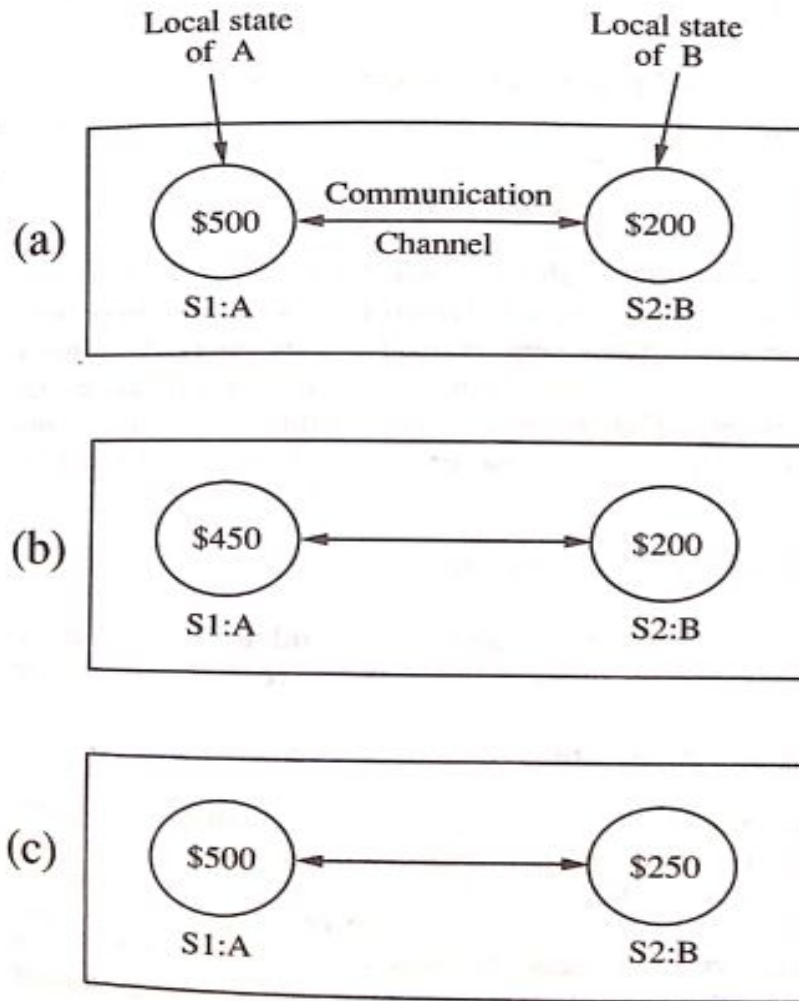
Fundamental Limitations of a Distributed System

- Distributed systems have **no shared memory**
 - Impact : Harder to collect up-to-date information.

Fundamental Limitations of a Distributed System

Example 5.1. Let S1 and S2 be two distinct sites (entities) of a distributed system (see Fig. 5.1) that maintain bank accounts A and B, respectively. A site in our example refers to a process. Knowledge of the global state of the system may be necessary to compute the net balance of both accounts. The initial state of the two accounts is shown in Fig. 5.1(a). Let site S1 transfer, say, \$50 from account A to account B. During the collection of a global state, if site S1 records the state of A immediately after the debit has occurred, and site S2 saves the state of B before the fund transfer message has reached B, then the global system state will show \$50 missing (see Fig. 5.1(b)). Note that the communication channel cannot record its state by itself. Hence, sites have to coordinate their state recording activities in order to record the channel state. On the other hand, if A's state is recorded immediately before the transfer and B's state is recorded after account B has been credited \$50, then the global system state will show an extra \$50 (see Fig. 5.1(c)).

Fundamental Limitations of a Distributed System



Example 5.1. Let S1 and S2 be two distinct sites (entities) of a distributed system (see Fig. 5.1) that maintain bank accounts A and B, respectively. A site refers to a process. Knowledge of the global state of the system may be used to compute the net balance of both accounts. The initial state of the two accounts is in Fig. 5.1(a). Let site S1 transfer, say, \$50 from account A to account B. To collect the collection of a global state, if site S1 records the state of A immediately after the debit has occurred, and site S2 saves the state of B before the funds have reached B, then the global system state will show \$50 missing (see Fig. 5.1(b)). Note that the communication channel cannot record its state by itself. The sites must have to coordinate their state recording activities in order to record the global state. On the other hand, if A's state is recorded immediately before the funds are transferred and the state is recorded after account B has been credited \$50, then the global state will show an extra \$50 (see Fig. 5.1(c)).

FIGURE 5.1
A distributed system with two sites.

Three Important Schemes

- Lamport's logical clocks
 - clock is a single integer
- Vector clocks
 - clock is a vector of integers with length n
 - n = number of processes
- Matrix clocks
 - clock is an $n \times n$ matrix of integers
 - n = number of processes

Lamport's Clock/ Scalar Clocks

HAPPENED BEFORE RELATION ($\rightarrow$). The *happened before* relation captures the causal dependencies between events, i.e., whether two events are causally related or not. The relation $\rightarrow$ is defined as follows:

- $a \rightarrow b$, if a and b are events in the same process and a occurred before b .
- $a \rightarrow b$, if a is the event of sending a message m in a process and b is the event of receipt of the same message m by another process.
- If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$, i.e., " $\rightarrow$ " relation is transitive.

In distributed systems, processes interact with each other and affect the outcome of events of processes. Being able to ascertain order between events is very important for designing, debugging, and understanding the sequence of execution in distributed computation. In general, an event changes the system state, which in turn influences the occurrence and outcome of future events. That is, past events influence future events and this influence among causally related events (those events that can be ordered by ' $\rightarrow$ ') is referred to as *causal affects*.

CAUSALLY RELATED EVENTS. Event a causally affects event b if $a \rightarrow b$.

CONCURRENT EVENTS. Two distinct events a and b are said to be concurrent (denoted by $a \parallel b$) if $a \not\rightarrow b$ and $b \not\rightarrow a$. In other words, concurrent events do not causally affect each other.

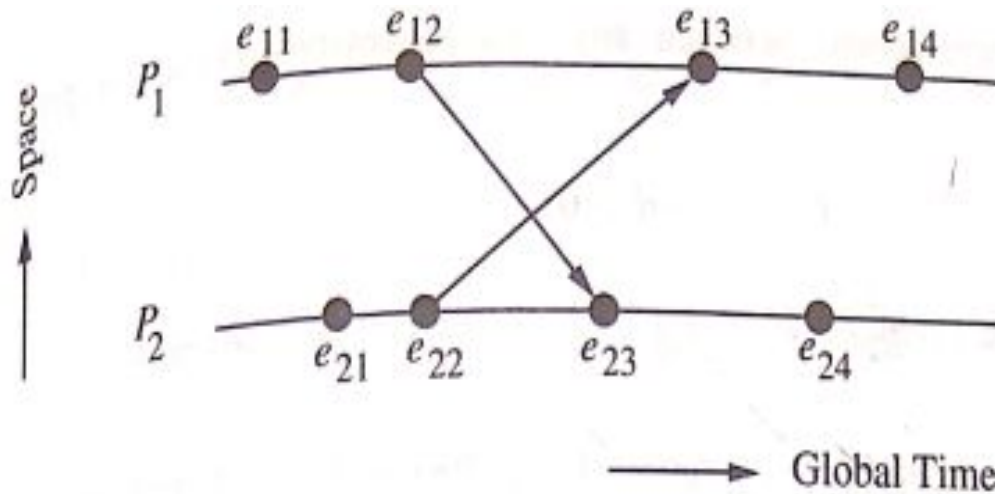
For any two events a and b in a system, either $a \rightarrow b$, $b \rightarrow a$, or $a \parallel b$.

Example 5.2 In the ...

Lamport's Clock/ Scalar Clocks

- Happened before relation:
 - $a \rightarrow b$: Event a occurred before event b. Events in the same process.
 - $a \rightarrow b$: If a is the event of sending a message m in a process and b is the event of receipt of the same message m by another process.
 - $a \rightarrow b, b \rightarrow c$, then $a \rightarrow c$. " $\rightarrow$ " is transitive.
- Causally Ordered Events
 - $a \rightarrow b$: Event a "causally" affects event b
- Concurrent Events
 - Only one is true: $a \rightarrow b$ (or) $b \rightarrow a$ (or) $a || b$.

Lamport's Clock- Example



Write all causally related and transitive events ?

Example 5.2. In the space-time diagram of Fig. 5.2, e_{11} , e_{12} , e_{13} , and e_{14} are events in process P_1 and e_{21} , e_{22} , e_{23} , and e_{24} are events in process P_2 . The arrows represent message transfers between the processes. For example, arrow $e_{12}e_{23}$ corresponds to a message sent from process P_1 to process P_2 , e_{12} is the event of sending the message at P_1 , and e_{23} is the event of receiving the same message at P_2 . In Fig. 5.2, we see that $e_{22} \rightarrow e_{13}$, $e_{13} \rightarrow e_{14}$, and therefore $e_{22} \rightarrow e_{14}$. In other words, event e_{22} causally

Lamport's logical clocks

Logical Clocks

In order to realize the relation $\rightarrow$, Lamport [12] introduced the following system of logical clocks. There is a clock C_i at each process P_i in the system. The clock C_i can be thought of as a function that assigns a number $C_i(a)$ to any event a , called the *timestamp* of event a , at P_i . The numbers assigned by the system of clocks have no relation to physical time, and hence the name logical clocks. The logical clocks take monotonically increasing values. These clocks can be implemented by counters. Typically, the timestamp of an event is the value of the clock when it occurs.

CONDITIONS SATISFIED BY THE SYSTEM OF CLOCKS. For any events a and b :

if $a \rightarrow b$, then $C(a) < C(b)$

The happened before relation ' $\rightarrow$ ' can now be realized by using the logical clocks if the following two conditions are met:

[C1] For any two events a and b in a process P_i , if a occurs before b , then

$$C_i(a) < C_i(b)$$

[C2] If a is the event of sending a message m in process P_i and b is the event of receiving the same message m at process P_j , then

$$C_i(a) < C_j(b)$$

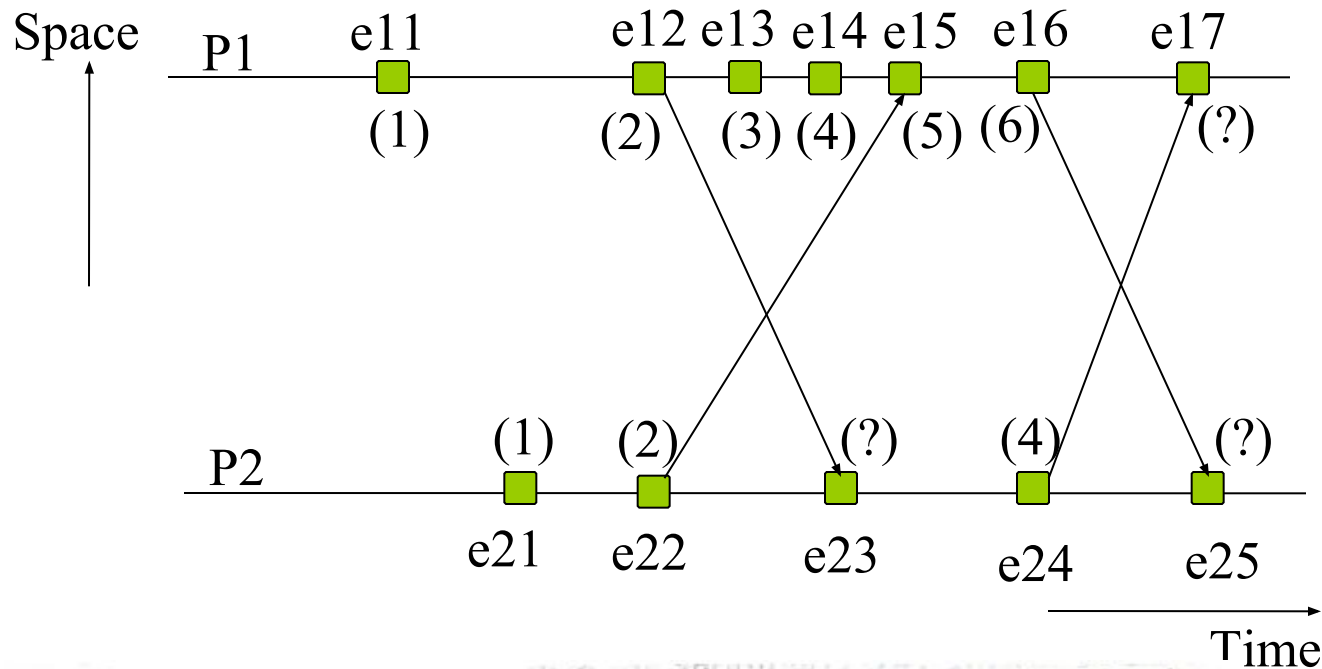
Logical Clocks

- Implementation Rules:
 - **R1: $C_i = C_i + d$ ($d > 0$);** clock is updated between two successive events at a process.
 - **R2: $C_j = \max(C_j, t_m + d)$; ($d > 0$);**
When P_j receives a message m with a time stamp t_m (t_m assigned by P_i , the sender; $t_m = C_i(a)$, a being the event of sending message m).

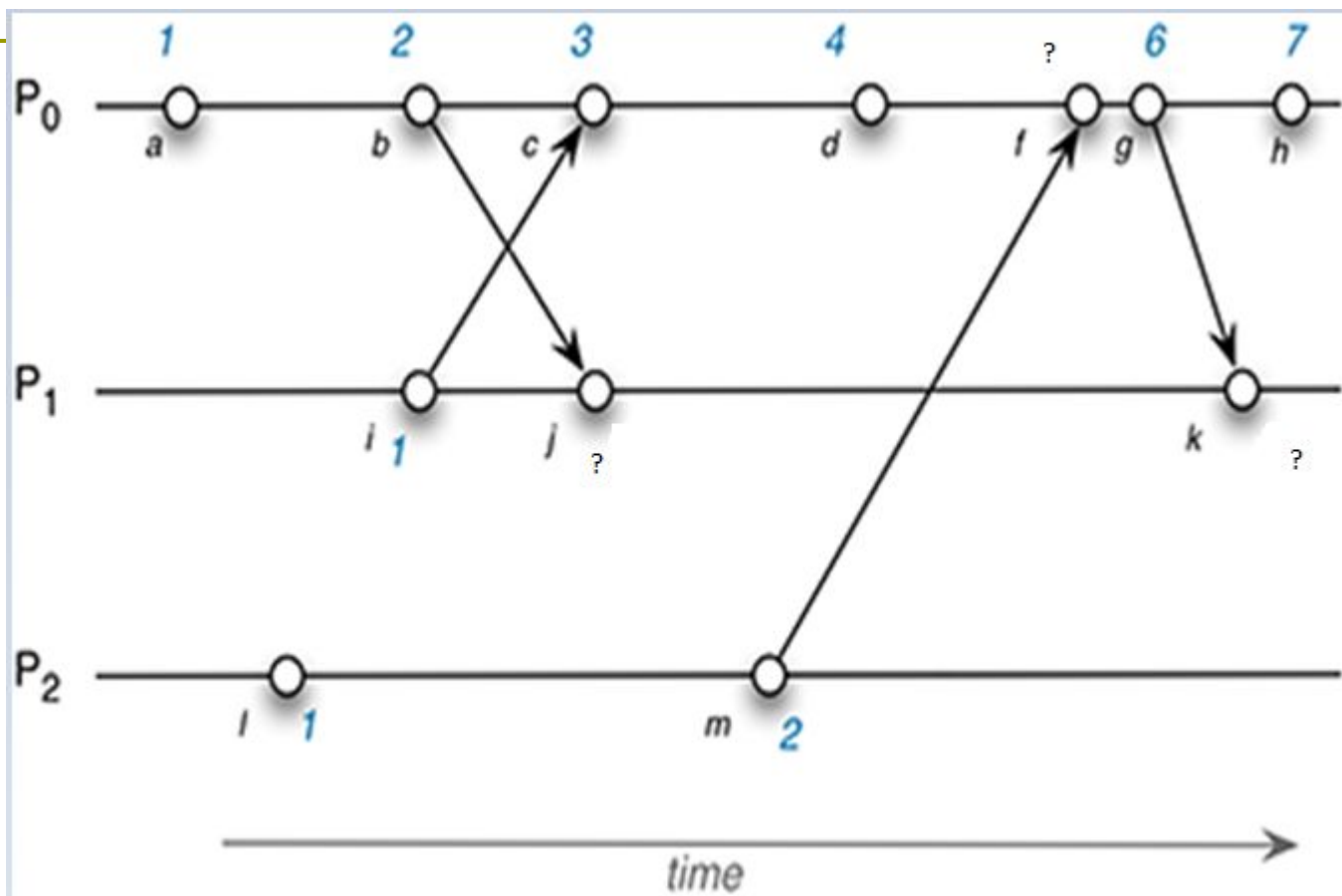
Logical Clocks

- Conditions satisfied:
 - C_i is clock in Process P_i .
 - If $a \rightarrow b$ in process P_i , $C_i(a) < C_i(b)$
 - a : sending message m in P_i ; b : receiving message m in P_j ; then, $C_i(a) < C_j(b)$.

Space-time Diagram



Example 5.3. Figure 5.3 gives an example of how logical clocks are updated under Lamport's scheme. Both the clock values C_{P_1} and C_{P_2} are assumed to be zero initially and d is assumed to be 1. e_{11} is an internal event in process P_1 which causes C_{P_1} to be incremented to 1 due to IR1. Similarly, e_{21} and e_{22} are two events in P_2 resulting in $C_{P_2} = 2$ due to IR1. e_{16} is a message send event in P_1 which increments C_{P_1} to 6 due to IR1. The message is assigned a timestamp = 6. The event e_{25} , corresponding to the receive event of the above message, increments the clock C_{P_2} to 7 ($\max(4+1, 6+1)$) due to rules IR1 and IR2. Similarly, e_{24} is a send event in P_2 . The message is assigned a timestamp = 4. The event e_{17} corresponding to the receive event of the above message increments the clock C_{P_1} to 7 ($\max(6+1, 4+1)$) due to rules IR1 and IR2.



Limitation of Lamport's Clock

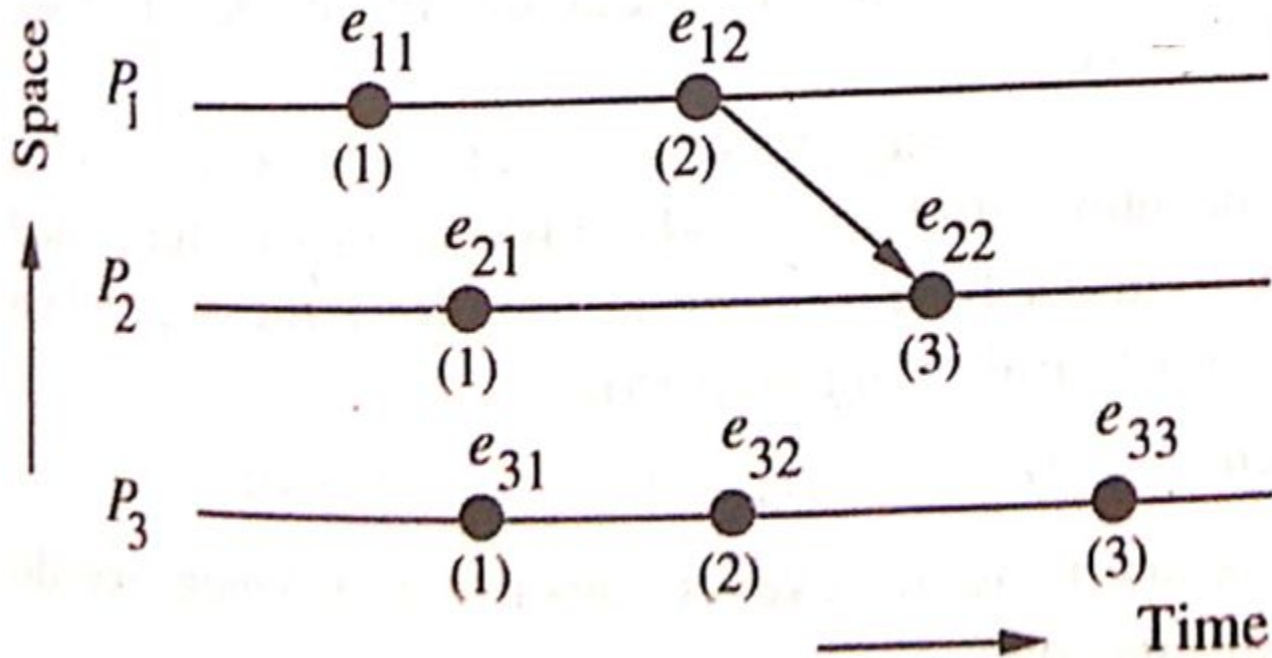
$a \rightarrow b$ implies $C(a) < C(b)$

BUT

$C(a) < C(b)$ doesn't imply $a \rightarrow b$!!

So not a true clock !!

Limitation of Lamport's Clock



a) Write all causally related events

b) Write those events which are not causally related to each other

Vector Time

- Vector time correctly captures the causality relationship between distributed events.
- Vector clock gives the ability to decide whether two events are casually related or not by simply looking at the timestamp.

Vector Time: Updates

The implementation rules for the vector clocks are as follows [16]:

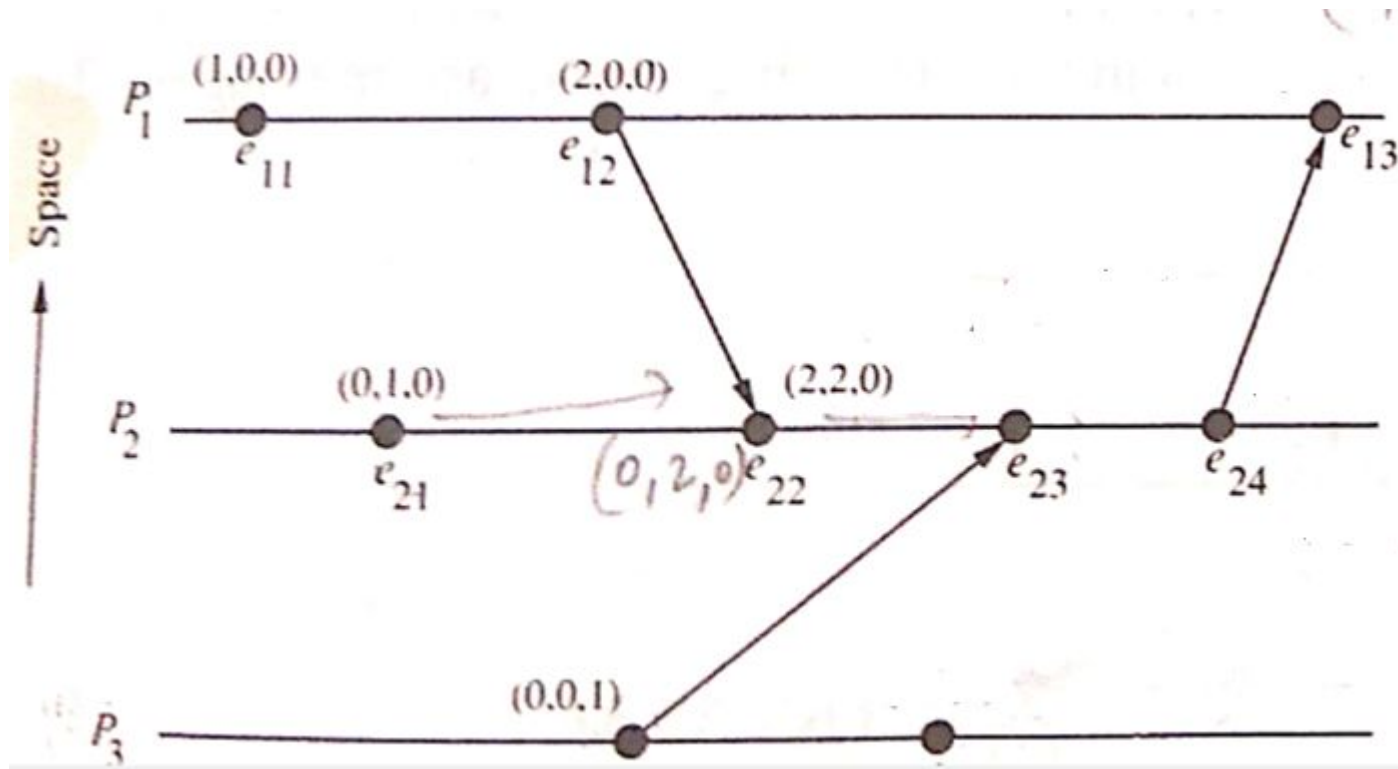
[IR1] Clock C_i is incremented between any two successive events in process P_i

$$C_i[i] := C_i[i] + d \quad (d > 0) \quad (5.3)$$

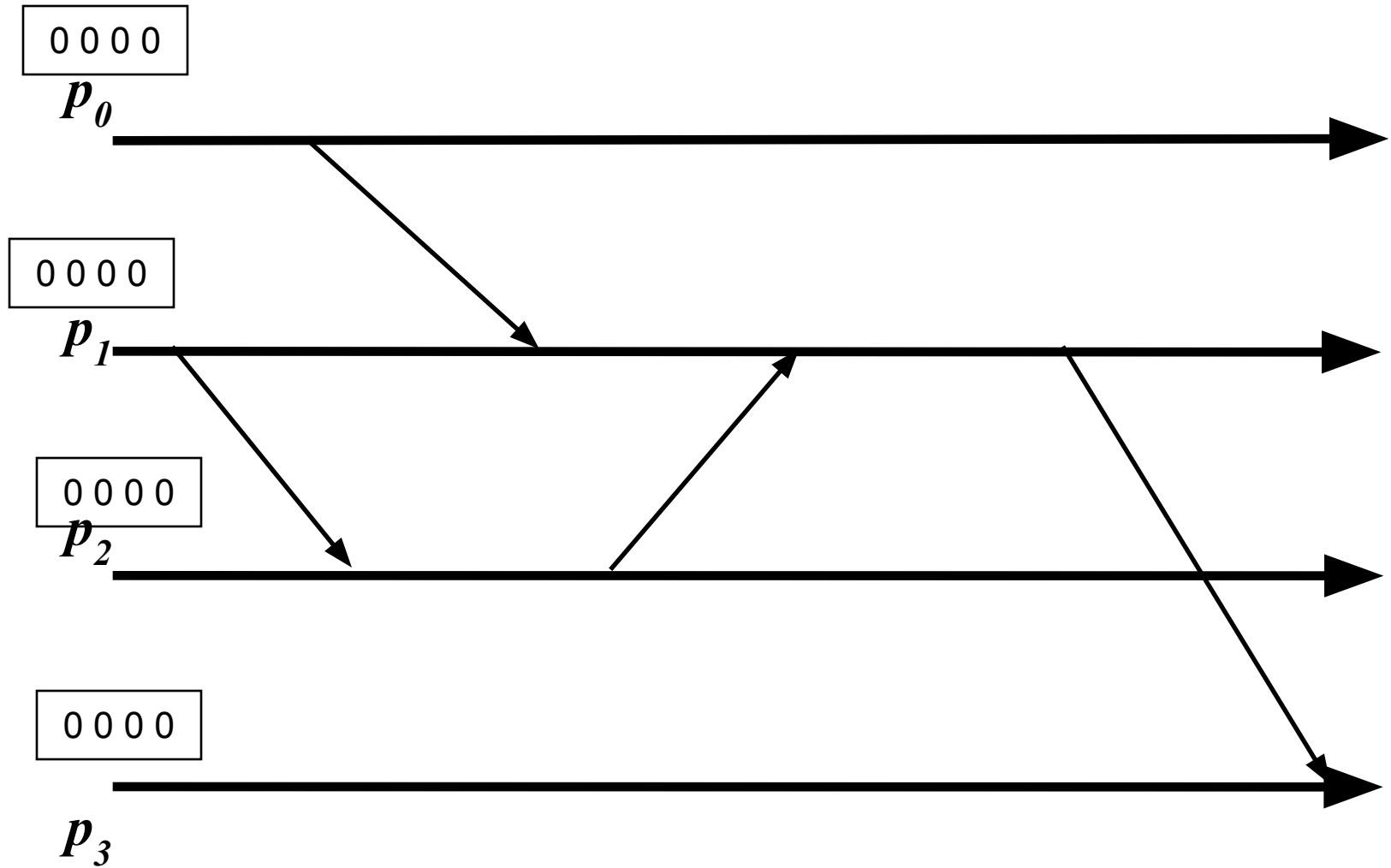
[IR2] If event a is the sending of the message m by process P_i , then message m is assigned a vector timestamp $t_m = C_i(a)$; on receiving the same message m by process P_j , C_j is updated as follows:

$$\forall k, C_j[k] := \max(C_j[k], t_m[k]) \quad (5.4)$$

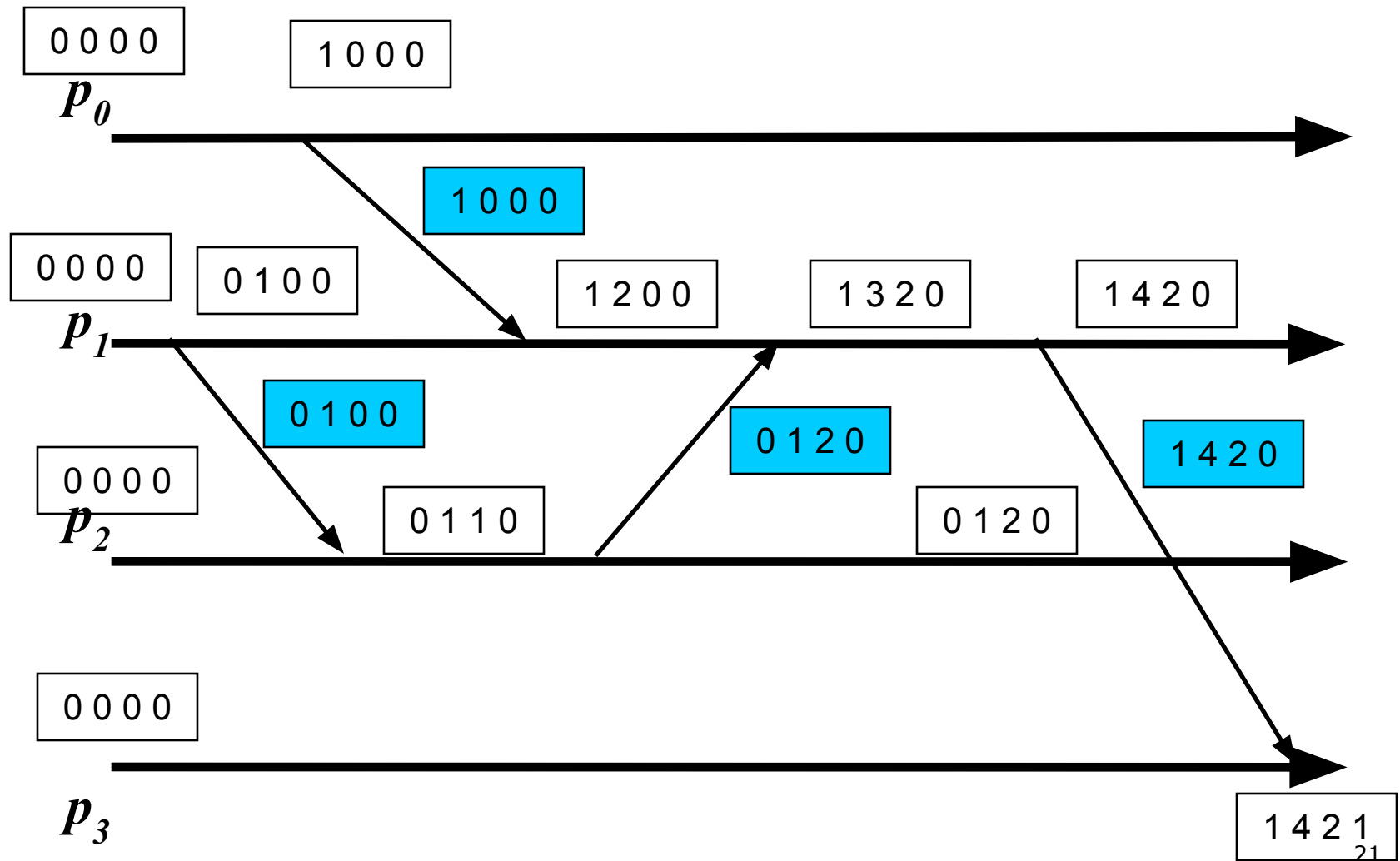
Vector Time: Updates



Vector Time: The Picture



Vector Time: The Picture



Comparison of vector timestamp

Vector timestamps can be compared as follows [16]. For any two vector timestamps t^a and t^b of events a and b , respectively:

Equal:	$t^a = t^b$	iff	$\forall i, t^a[i] = t^b[i];$
Not Equal:	$t^a \neq t^b$	iff	$\exists i, t^a[i] \neq t^b[i];$
Less Than or Equal:	$t^a \leq t^b$	iff	$\forall i, t^a[i] \leq t^b[i];$
Not Less Than or Equal To:	$t^a \not\leq t^b$	iff	$\exists i, t^a[i] > t^b[i];$
Less Than:	$t^a < t^b$	iff	$(t^a \leq t^b \wedge t^a \neq t^b);$
Not Less Than:	$t^a \not< t^b$	iff	$\neg(t^a \leq t^b \wedge t^a \neq t^b);$
Concurrent:	$t^a \parallel t^b$	iff	$(t^a \not< t^b \wedge t^b \not< t^a);$

Give some examples

How can vector clocks help in causal ordering of messages ?

Delivery in Causal Order:

If $\text{send}(m1) \rightarrow \text{send}(m2)$, then every recipient of both messages $m1$ and $m2$ must “deliver” $m1$ before $m2$

{“deliver” – when the message is actually given to the application for processing}

Causal Order of Messages

Techniques for the causal ordering of messages are useful in developing distributed algorithms and may simplify the algorithms themselves. For example, for applications such as replicated database systems, it is important that every process in charge of updating a replica receives the updates in the same order to maintain the consistency of the database [1]. In the absence of causal ordering of messages, each and every update must be checked to ensure that it does not violate the consistency constraints.

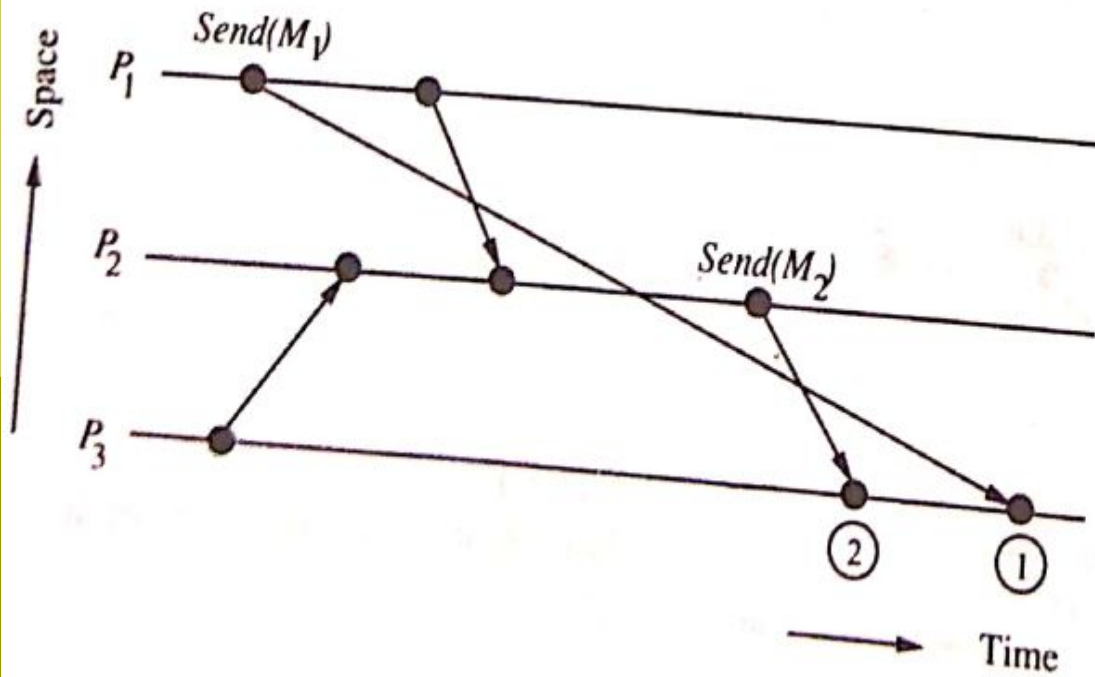


FIGURE 5.6
An example of the violation of causal ordering of messages.

Birman-Schiper-Stephenson Protocol

- (i) To broadcast m from process i , increment $C_i(i)$, and timestamp m with $VT_m = C_i[i]$
- (ii) When $j \neq i$ receives m , j delays delivery of m until
 1. $C_j[i] = VT_m[i] - 1$ and
 2. $C_j[k] \geq VT_m[k]$ for all $k \neq i$
 1. says m is the next msg P_j was expecting from P_i
 2. says P_j has seen all msgs seen by P_i when P_i sent m .

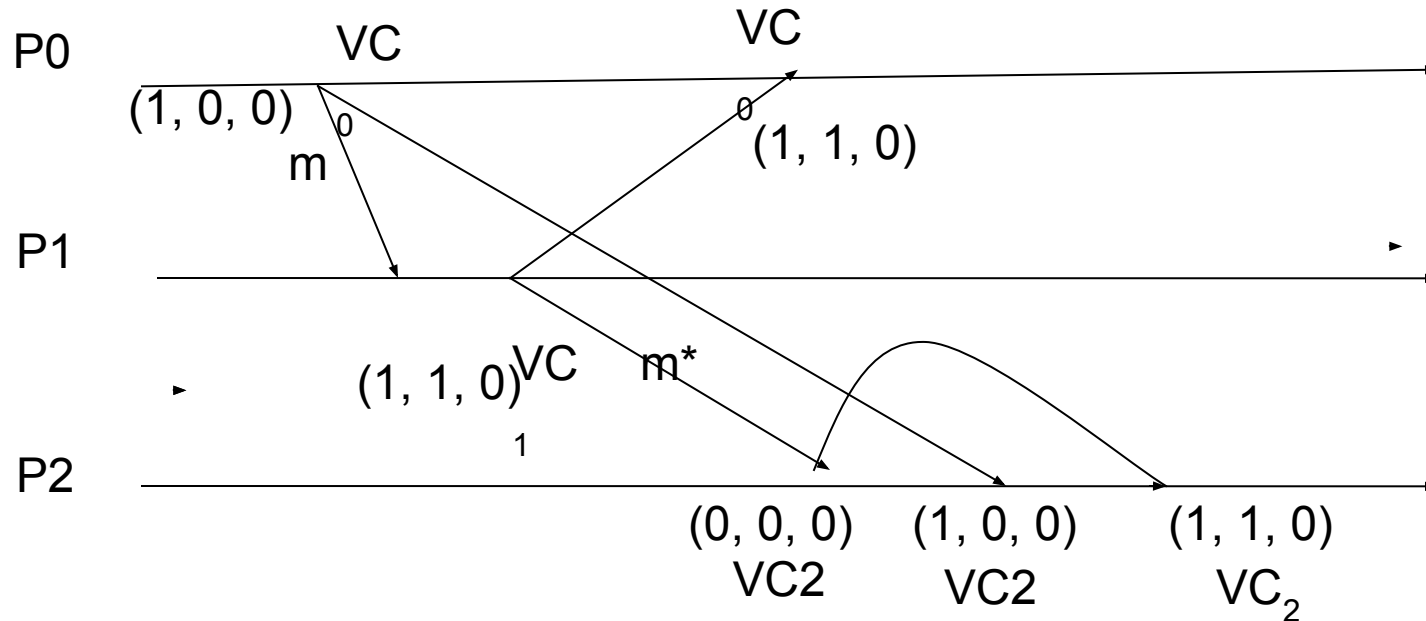
Delayed messages are queued in j sorted by vector time

Clocks are updated only when messages are sent.

Causal Message ordering(contd)

- **If a message m is received from P_i , you should also have received every message that P_i received before it sent m ; e.g.,**
 - if m is sent by P_1 and $ts(m)$ is $(3, 4, 0)$ and you are P_3 , you should already have received exactly 2 messages from P_1 and at least 4 from P_2
 - if m is sent by P_2 and $ts(m)$ is $(4, 5, 1, 3)$ and if you are P_3 and VC_3 is $(3, 3, 4, 3)$ then you need to wait for a fourth message from P_2 and at least one more message from P_1 .

Enforcing Causal Communication



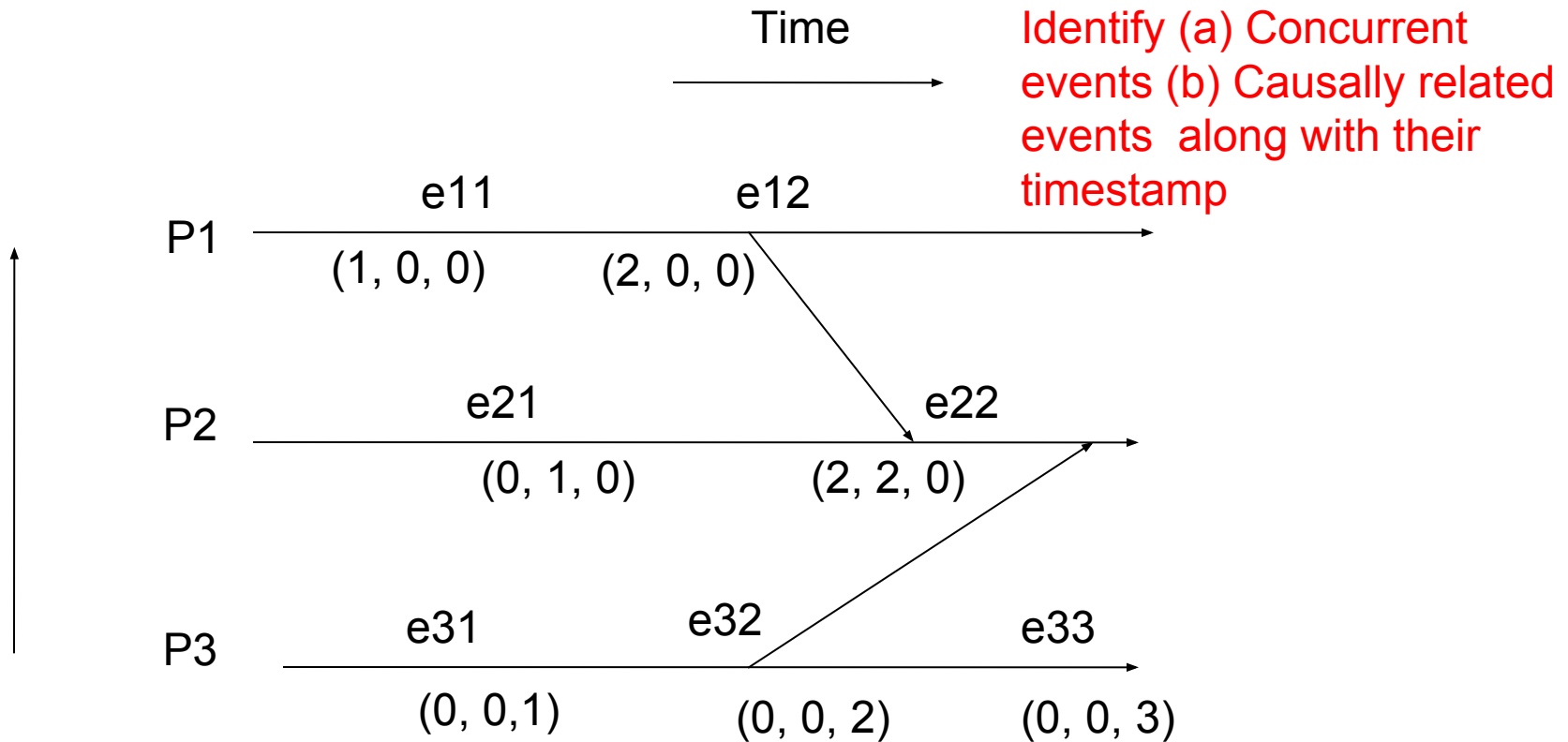
P1 received message m from P0 before sending message m^* to P2; P2 must wait for delivery of m before receiving m^*

(Increment own clock only on message send)

Before sending or receiving any messages, one's own clock is (0, 0, ...0)

Vector Time: Comparison

- Vector timestamps are compared in the following manner:
 - $TS_i < TS_j$ iff
$$\forall k, TS_i[k] \leq TS_j[k] \text{ and } \exists k, TS_i[k] < TS_j[k].$$
eg [(3,3,5) < (3,4,5)]
 - An event e causally depends on an event e' iff
$$e'.TS < e.TS$$
 - Two events e and e' are concurrent if neither $e'.TS < e.TS$ nor $e.TS < e'.TS$



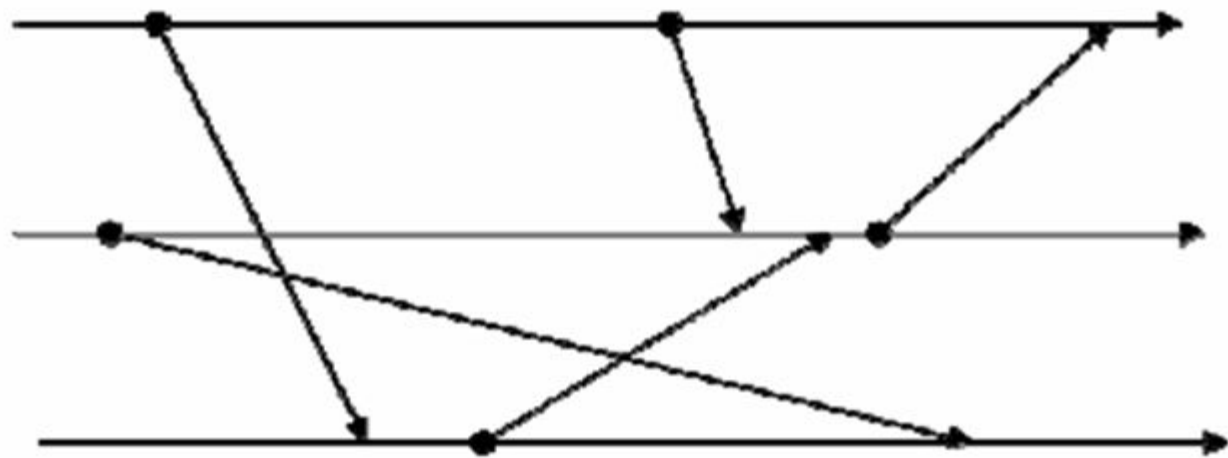
$ts(e_{11}) = (1, 0, 0)$ and $ts(e_{32}) = (0, 0, 2)$, which shows that the two events are concurrent.

$ts(e_{11}) = (1, 0, 0)$ and $ts(e_{22}) = (2, 2, 0)$, which shows that $e_{11} \longrightarrow e_{22}$

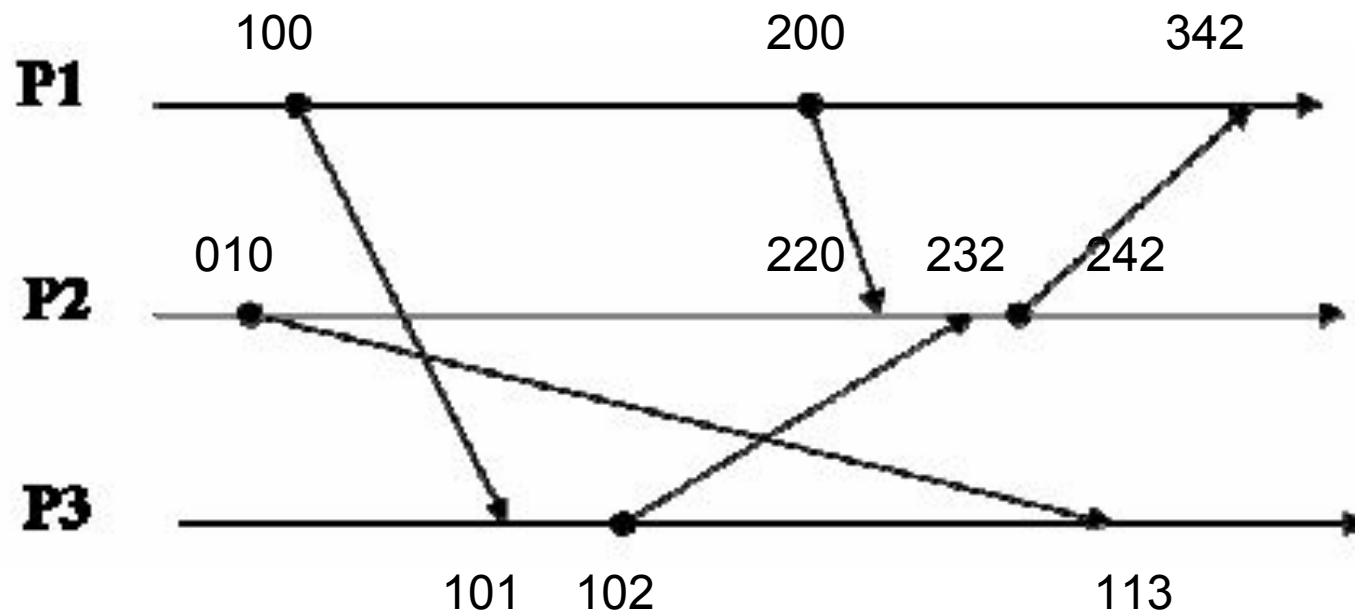
P1

P2

P3



Compute the vector that is piggybacked on each message.



Matrix Clocks

□ Matrix clock:

- Each event has n vector clocks, one for each process
- The i th vector on process i is called process i 's principle vector i.e. its own vector clock as before

Matrix Time

In a system of matrix clocks, the time is represented by a set of $n \times n$ matrices of non-negative integers.

->Vector clock :

one-dimensional data structure

->Matrix clock :

Two-dimensional data structure

-> Message and computation overheads are high

-> Powerful and useful for applications in distributed system

Matrix Time

In a system of matrix clocks, the time is represented by a set of $n \times n$ matrices of non-negative integers.

A process p_i maintains a matrix $mt_i[1..n, 1..n]$ where,

- **$mt_i[i, i]$** denotes the local logical clock of p_i and tracks the progress of the computation at process p_i .
- **$mt_i[i, j]$** denotes the latest knowledge that process p_i has about the local logical clock, $mt_j[j, j]$, of process p_j .
- **$mt_i[j, k]$** represents the knowledge that process p_i has about the latest knowledge that p_j has about the local logical clock, $mt_k[k, k]$, of p_k .

The entire matrix mt_i denotes p_i 's local view of the global logical time.

Process p_i uses the following rules R1 and R2 to update its clock:

- R1 : Before executing an event, process p_i updates its local logical time as follows:

$$\mathbf{mt_i[i , i] := mt_i[i , i] + d \ (d > 0)}$$

- R2: Each message m is piggybacked with matrix time mt . When p_i receives such a message (m, mt) from a process p_j , p_i executes the following sequence of actions:

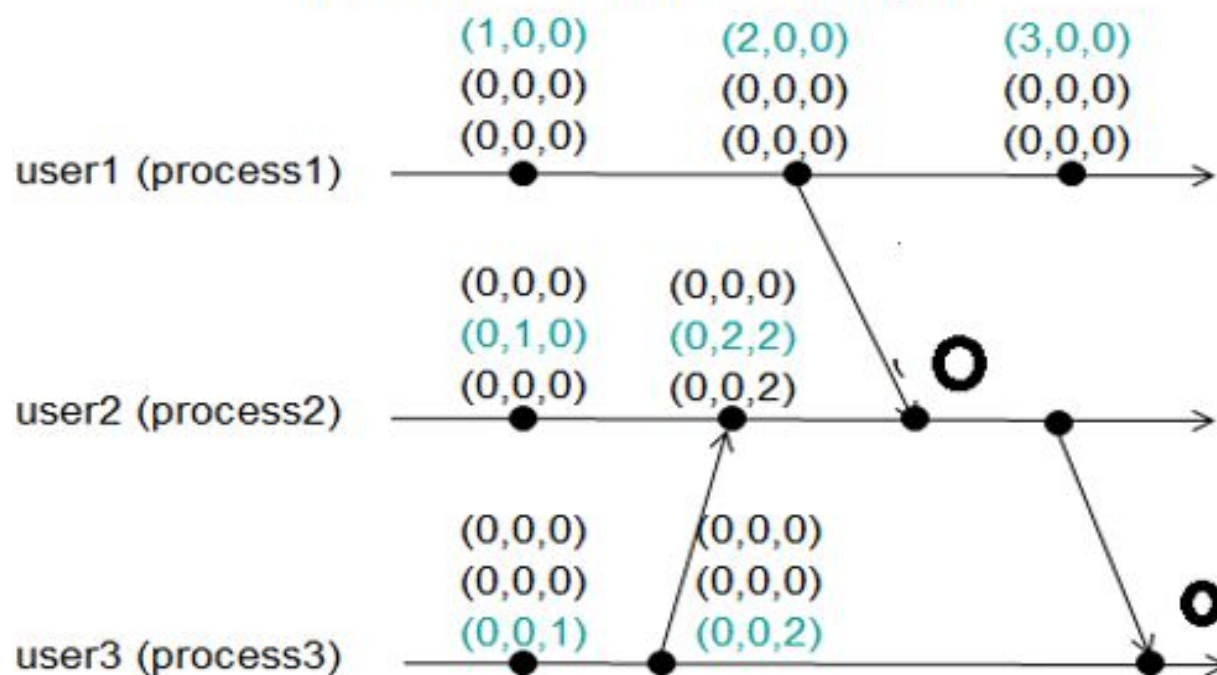
Update its global logical time as follows:

$$\text{(a) } \mathbf{1 \leq k \leq n : mt_i[i , k] := \max(mt_i[i , k], mt[j , k])}$$

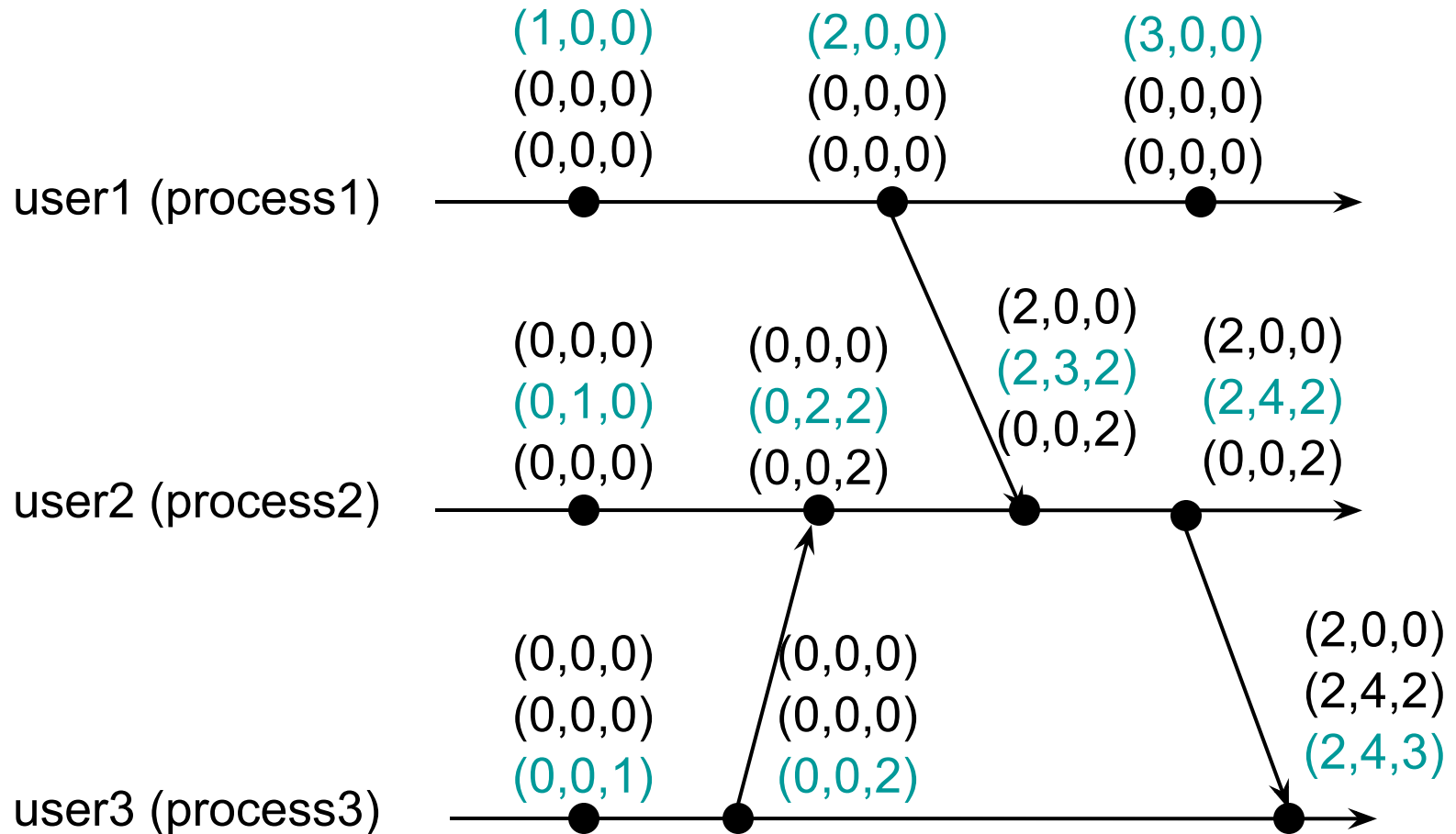
$$\text{(b) } \mathbf{1 \leq k, l \leq n : mt_i[k, l] := \max(mt_i[k, l], mt[k, l])}$$

- Execute R1.
- Deliver message m .

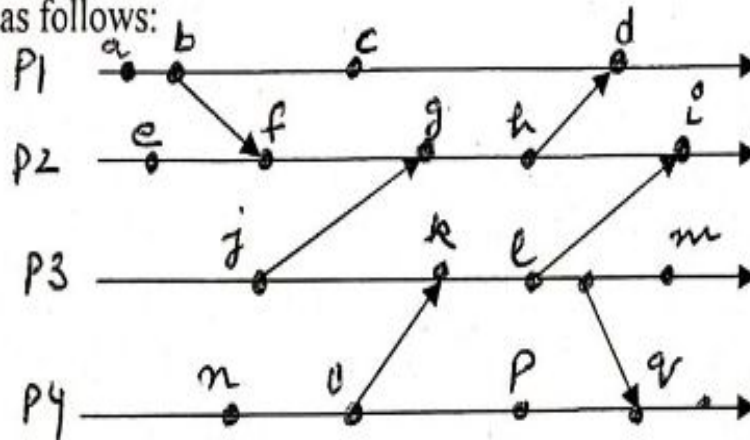
Matrix Clock Example



Matrix Clock Example



Q1. [CO1 3+ 1 marks] a) Calculate the matrix clock timestamps for each event in the system with 4 processes as follows:



b) Show four events (if they exist) in the above diagram with their timestamps for which Lamport's clock cannot capture their causality relationship.

Compute the matrix clock values

